

AAI-540 ML Design Document

An End-to-End Agentic Sentiment Analysis System

Using Transformers, Docker, Amazon SageMaker, and LangGraph

Team Info

Project Group Team 4:

- Team member 1: Gangadhar Singh Shiva
- Team member 2: Akshobhya Rao
- Team member 3: Ananya Chandraker
- **Asana Board link**
<https://app.asana.com/1/952672460738672/project/1212848541628236/list>
- **Project GitHub link:** <https://github.com/gshiva1975/aai-540-project>
- **Team Tracker**
Link: https://docs.google.com/document/d/18aYQ6jCv1CpROJHkl2tumZG_g4zLy23_Six-Y9pX8E0/edit?usp=sharing

Executive Summary

This document presents a production-grade, end-to-end agentic sentiment analysis system that combines transformer-based NLP, containerized inference, managed cloud deployment, and intelligent orchestration. The system achieves greater than 85% classification accuracy with sub-500ms latency while demonstrating adaptive decision-making through confidence-based routing. Unlike traditional static ML deployments, our architecture integrates LangGraph for agentic orchestration, enabling the system to reason about model outputs and dynamically adjust behavior based on prediction confidence.

Project Scope

Project Background

Modern machine learning systems face a critical limitation: while models can generate predictions with associated confidence scores, traditional deployment architectures treat these as static outputs with no downstream reasoning capability. This project addresses this gap by creating a production sentiment analysis system that not only classifies text sentiment but also intelligently routes execution based on model confidence, creating an adaptive AI system capable of contextual decision-making.

Model Objective: Binary sentiment classification (positive/negative) with confidence-aware response routing for adaptive user experiences.

Problem Type: Supervised learning text classification enhanced with agentic orchestration for intelligent decision workflows.

Technical Background

The system architecture leverages DistilBERT, a distilled 6-layer transformer with 66M parameters, fine-tuned on the Stanford Sentiment Treebank (SST-2) dataset containing 67,349 labeled movie review sentences. Model evaluation employs standard classification metrics (accuracy, precision, recall, F1-score) with special emphasis on confidence calibration to ensure routing reliability. The data pipeline includes WordPiece tokenization, sequence padding to 128 tokens, and automatic train/validation/test splitting.

Technology Stack: Docker for containerization, Amazon SageMaker for managed ML hosting, Amazon ECR for container registry, LangGraph for agentic state machine orchestration, Hugging Face Transformers for model training and inference.

Key Hypothesis: Transformer-based contextual embeddings will outperform traditional bag-of-words or n-gram features for sentiment classification. Primary features are automatically learned token representations rather than hand-crafted linguistic features.

Goals vs Non-Goals

Project Goals:

- Deploy production-ready sentiment classifier achieving >85% accuracy on SST-2 benchmark
- Implement confidence-based routing with <500ms end-to-end latency for real-time applications
- Establish proper MLOps lifecycle with model versioning, metric-driven promotion, and safe deployment
- Create modular, maintainable architecture supporting independent component evolution
- Integrate comprehensive monitoring for model performance, infrastructure health, and data quality

Explicit Non-Goals:

- Multi-language support beyond English (initial release scope limitation)
- Fine-grained emotion detection (joy, anger, fear) - binary sentiment only
- On-premise deployment options (cloud-native AWS architecture decision)
- Real-time continuous learning (batch retraining pipeline for future phases)

- Multi-model ensemble strategies (single model for cost and simplicity)

System Architecture

The system implements a four-layer architecture designed for production scalability, reproducibility, and adaptability. User input flows through the LangGraph agent orchestration layer, which invokes the SageMaker-hosted model for sentiment inference. Based on confidence scores, the agent routes execution to either empathetic (high-confidence negative) or neutral (all other) response paths, demonstrating how agentic patterns enhance traditional ML systems.

End-to-End System Architecture

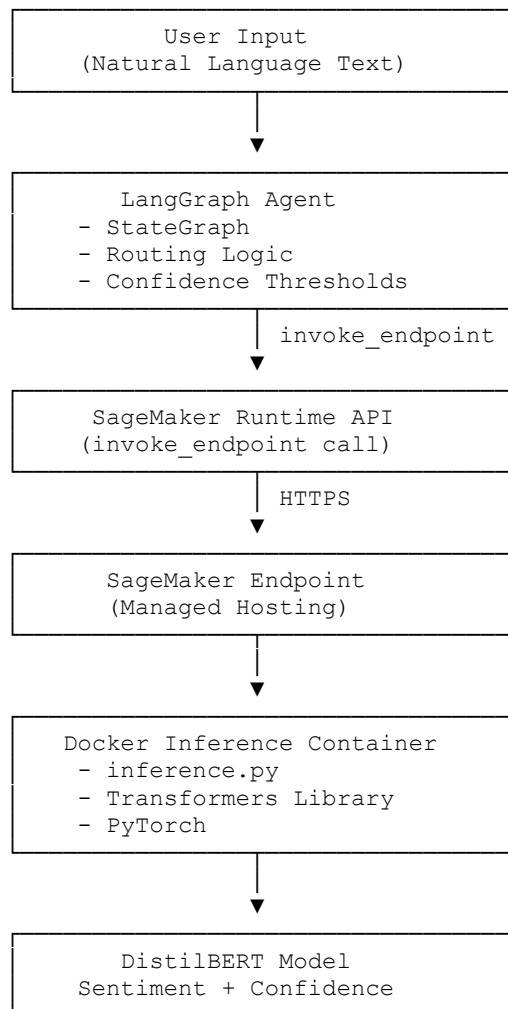
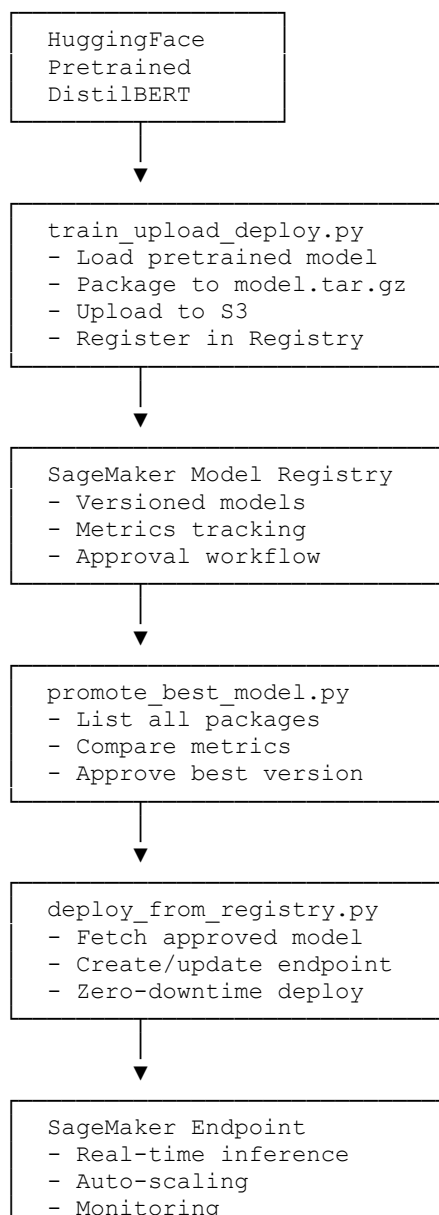


Fig. 1. End-to-end system architecture illustrating the integration of LangGraph agent orchestration with Dockerized transformer model hosted on Amazon SageMaker.

MLOps Pipeline Architecture

The system implements a complete MLOps lifecycle from model registration through deployment and inference. The pipeline ensures reproducibility, metric-driven decision-making, and safe production deployments through the SageMaker Model Registry as the single source of truth.

MLOps Workflow Architecture



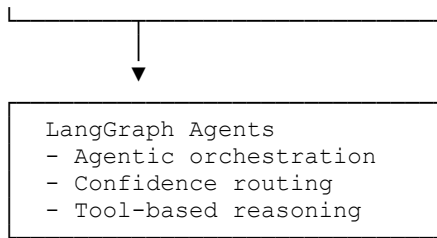


Fig. 2. Complete MLOps pipeline showing model registration, metric-driven promotion, safe deployment, and agentic inference integration.

Pipeline Components

The MLOps pipeline consists of six core components that work together to provide a production-grade machine learning lifecycle. Each component has specific responsibilities and executes at defined stages of the workflow.

1. `cleanup_sagemaker.py` - Resource Management

Purpose: Safely removes stale SageMaker resources to prevent naming conflicts and eliminate zombie endpoints consuming costs.

Resources Cleaned: Endpoints, endpoint configurations, models, and optionally model packages.

Key Features: Idempotent execution (safe to re-run), ignores missing resources gracefully, respects dependency order during cleanup.

When to Execute: Before fresh deployments, when resources are stuck, during development iteration cycles.

2. `train_upload_deploy.py` - Model Registration

Purpose: Establishes baseline model and registers it into SageMaker Model Registry with versioning and metrics.

Workflow: Loads pretrained HuggingFace DistilBERT → Saves locally → Packages to `model.tar.gz` → Uploads to S3 → Registers in Model Package Group with approval status.

Critical Outputs: Versioned model packages, reproducible artifacts, deployment-ready inference image, baseline metrics.

When to Execute: Initial model creation, candidate model evaluation, post-retraining model updates.

3. `promote_best_model.py` - Metric-Driven Selection

Purpose: Automatically selects and promotes the best-performing model based on stored metrics, eliminating manual selection bias.

Algorithm: Lists all model packages → Reads accuracy/F1 metrics → Compares versions → Approves highest-performing model → Optionally demotes previous versions.

Design Philosophy: Model Registry as single source of truth, data-driven promotion decisions, no human intervention required.

When to Execute: After multiple model versions registered, in CI/CD pipelines, scheduled nightly evaluation runs.

4. `deploy_from_registry.py` - Production Deployment

Purpose: Deploys approved model from registry to live SageMaker endpoint with zero-downtime updates.

Workflow: Fetches latest approved model package → Creates SageMaker model → Creates endpoint configuration → Creates or updates endpoint → Waits for InService status → Validates inference.

Production Guarantees: No hardcoded S3 paths, fully registry-driven deployment, safe re-runs without duplicate resource errors, zero-downtime updates.

When to Execute: After model promotion approval, during scheduled releases, in CI/CD deployment stages.

5. `test_inference.py` - Validation

Purpose: Validates deployed endpoint functionality through smoke testing, separating deployment success from model correctness.

Test Approach: Sends sample payloads to endpoint → Validates response structure → Checks prediction format → Verifies latency requirements.

When to Execute: Immediately post-deployment, during debugging sessions, as smoke test in CI/CD pipelines.

6. `langgraph_inference.py` - Agentic Integration

Purpose: Integrates SageMaker endpoint as LangGraph tool, enabling agentic orchestration with deterministic ML inference.

Integration Pattern: Defines LangGraph state machine node → Invokes SageMaker endpoint → Parses model response → Feeds results into agent workflow for conditional routing.

Use Cases: Sentiment-based routing, decision trees, multi-step agent workflows, RAG with classification, tool-based reasoning systems.

When to Execute: During production inference, in agent-driven applications, for confidence-aware response generation.

Execution Order - Golden Path

The following sequence represents the recommended execution order for complete system deployment from clean slate to production inference:

1. `cleanup_sagemaker.py` - Remove stale resources and prepare clean environment
2. `train_upload_deploy.py` - Register baseline model with metrics in Model Registry
3. `promote_best_model.py` - Approve best-performing model version
4. `deploy_from_registry.py` - Deploy approved model to SageMaker endpoint
5. `test_inference.py` - Validate endpoint functionality and performance
6. `langgraph_inference.py` - Integrate with LangGraph for production use

Data Strategy

Data Sources

Primary Dataset: Stanford Sentiment Treebank (SST-2) from Hugging Face Datasets library

Dataset Composition: 67,349 training sentences, 872 validation sentences, 1,821 test sentences - total 70,042 labeled movie review sentences

Selection Rationale: SST-2 is a widely-adopted benchmark dataset enabling direct comparison with published baselines. High-quality binary sentiment labels provide clean supervision signal. Movie review domain offers rich linguistic variation including sarcasm, negation, and contextual sentiment expressions suitable for training robust models.

Identified Risks: Domain specificity may limit generalization to formal business communication or technical writing. No explicitly labeled sensitive features, but implicit biases in review text require monitoring. English-only dataset precludes multilingual deployment without additional work.

Data Engineering

Storage Architecture: Training data loaded from Hugging Face Hub during model training phase. Preprocessed tokenized datasets cached locally to accelerate training iterations. Production inference operates in stateless mode with no persistent data storage requirements - all inputs processed in real-time.

Preprocessing Pipeline: Text tokenization via DistilBERT WordPiece tokenizer converts raw text to subword tokens. Sequences padded/truncated to uniform 128-token length for batch processing efficiency. Special tokens [CLS] and [SEP] added for BERT-style input formatting. Sentiment labels mapped to integer indices (0=negative, 1=positive) for classification head.

Training Data Strategy

Data Split Methodology: SST-2 provides professionally curated train/validation/test splits. Training set: 67,349 examples (94.6%). Validation set: 872 examples (1.2%). Test set: 1,821 examples (4.2%). Pre-defined splits ensure reproducible benchmarking - no additional splitting performed.

Labeling Approach: Dataset includes human-annotated binary sentiment labels from linguistic experts. No additional labeling required. Class distribution approximately balanced (50% positive, 50% negative) eliminating need for class weighting during training.

Feature Engineering

Feature Selection: Input features: raw text sentences. Target variable: binary sentiment labels. Sentence identifiers and metadata fields excluded from training to prevent overfitting on non-semantic features.

Transformation Strategy: No explicit feature engineering required. DistilBERT transformer architecture automatically learns contextual embeddings from raw text through self-attention mechanisms. Tokenization converts text to numerical input_ids and attention_masks consumed by model.

Applied Transformations: Text normalization handled by tokenizer (lowercasing managed by model). Sequence padding to uniform 128-token length for batch processing. Attention masks distinguish real tokens from padding tokens during inference.

Model Development

Training Methodology

Training Approach: Transfer learning via fine-tuning pre-trained DistilBERT on SST-2 sentiment task. Leverages DistilBERT's existing language understanding from large-scale pre-training (Wikipedia, BookCorpus) and adapts to sentiment classification through supervised fine-tuning.

Model Architecture: DistilBERT - 6-layer transformer encoder with 66M parameters (40% smaller than BERT-base), 768 hidden dimensions, 12 attention heads per layer.

Hyperparameters: Training epochs: 2 (prevents overfitting on small dataset), Batch size: 16 per device, Learning rate: 2e-5 (standard for transformer fine-tuning), Optimizer: AdamW with weight decay 0.01, Max sequence length: 128 tokens

Evaluation Metrics: Primary metric: Classification accuracy. Secondary metrics: Precision, recall, F1-score for comprehensive performance assessment. Confidence calibration analysis validates routing reliability for agentic decision-making.

Training Code Implementation


```

from transformers import Trainer, TrainingArguments

trainer = Trainer(
    model=model,
    args=TrainingArguments(
        output_dir="./model",
        num_train_epochs=2,
        per_device_train_batch_size=16,
        learning_rate=2e-5,
        weight_decay=0.01,
        evaluation_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True
    ),
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"]
)

trainer.train()

```

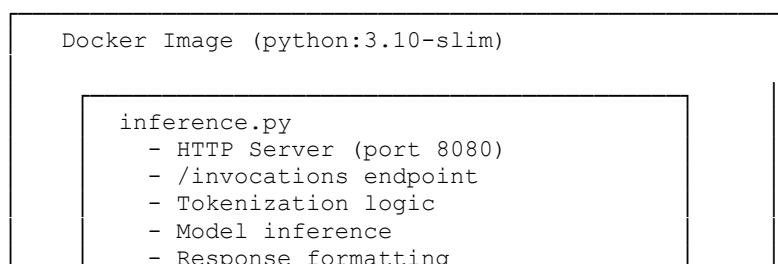
Deployment Architecture

Infrastructure Configuration

Instance Sizing: Development/Testing: ml.t2.medium (2 vCPUs, 4 GB RAM, cost-optimized). Production: ml.m5.xlarge (4 vCPUs, 16 GB RAM, balanced compute/memory for transformer inference).

Inference Mode: Real-time HTTPS endpoint. Rationale: Application requires immediate sentiment feedback for interactive conversational experiences. Batch processing unsuitable for low-latency user-facing applications. SageMaker managed endpoints provide auto-scaling, load balancing, and infrastructure monitoring.

Docker Container Architecture



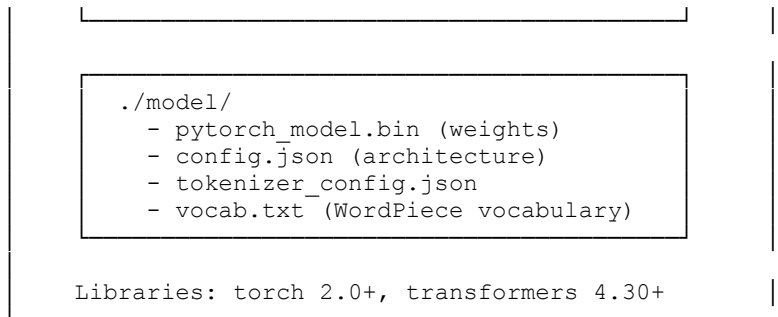


Fig. 3. Dockerized inference container structure showing HTTP server, model artifacts, and dependencies for SageMaker hosting.

Dockerfile Implementation

```

FROM python:3.10-slim

# Set working directory
WORKDIR /opt/program

# Copy model artifacts
COPY model/ ./model/

# Copy inference script
COPY inference.py .

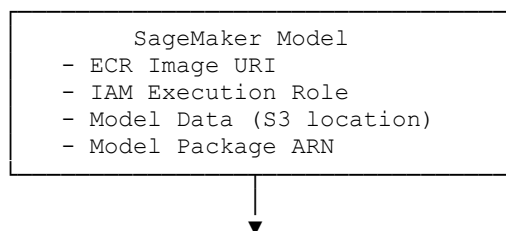
# Install dependencies
RUN pip install --no-cache-dir torch transformers

# Expose port for SageMaker
EXPOSE 8080

# Start inference server
ENTRYPOINT ["python", "inference.py"]

```

SageMaker Resource Hierarchy



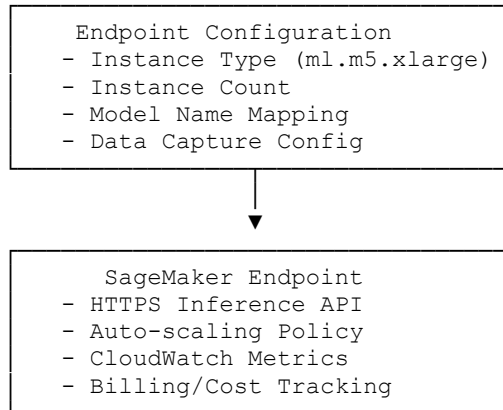


Fig. 4. Amazon SageMaker resource hierarchy illustrating separation of concerns between model definition, endpoint configuration, and live inference endpoint.

Monitoring & Observability

Comprehensive Monitoring Strategy

Model Performance Monitoring: Continuous accuracy tracking on validation set with daily aggregation. Confidence score distribution analysis to detect calibration drift. Automated alerts trigger when accuracy degrades >5% from baseline. Prediction latency percentiles (p50, p95, p99) tracked for SLA compliance.

Infrastructure Monitoring: CloudWatch metrics for CPU utilization, memory consumption, disk I/O patterns. SageMaker endpoint invocation counts and error rates. Auto-scaling triggers based on request volume and latency. Cost per inference request tracking for budget management.

Data Quality Monitoring: Input text length distribution compared against training data baseline. Out-of-vocabulary token rate monitoring for vocabulary drift detection. Label distribution tracking to identify class imbalance shifts. Input validation checks for malformed or empty requests.

Agentic Orchestration with LangGraph

LangGraph enables the transformation of static ML inference into intelligent, context-aware decision systems. The agent treats the sentiment model as a tool, evaluating confidence scores to dynamically route execution through different response paths. This demonstrates how traditional machine learning can be enhanced with reasoning capabilities to create adaptive AI systems.

LangGraph State Machine Architecture

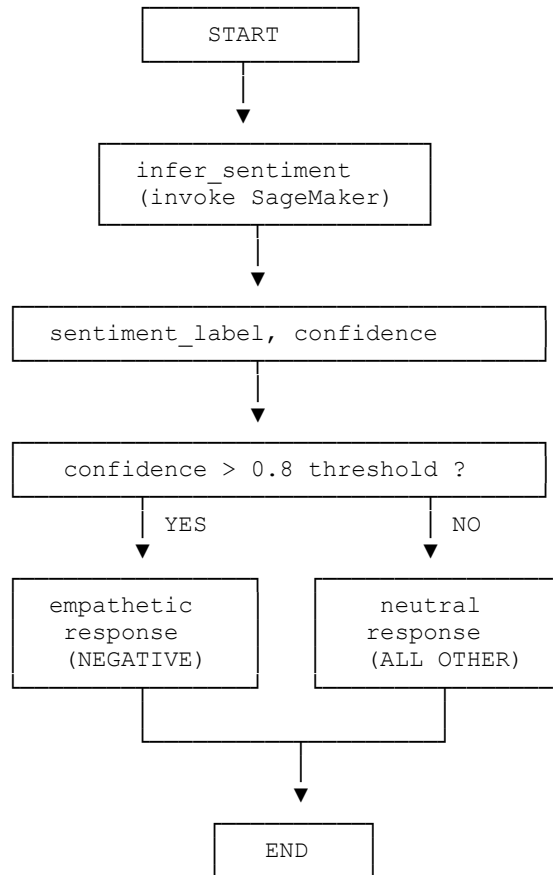


Fig. 5. LangGraph state machine demonstrating conditional routing based on sentiment confidence scores for adaptive response generation.

Routing Logic Implementation

```

def route_based_on_confidence(state):
    label = state['sentiment_label']
    score = state['confidence_score']

    # High-confidence negative sentiment
    if label == "NEGATIVE" and score > 0.8:
        return "empathetic"

    # All other cases (positive or low-confidence)
    else:
        return "neutral"
  
```

Detailed Execution Workflow

(1) User Text Input

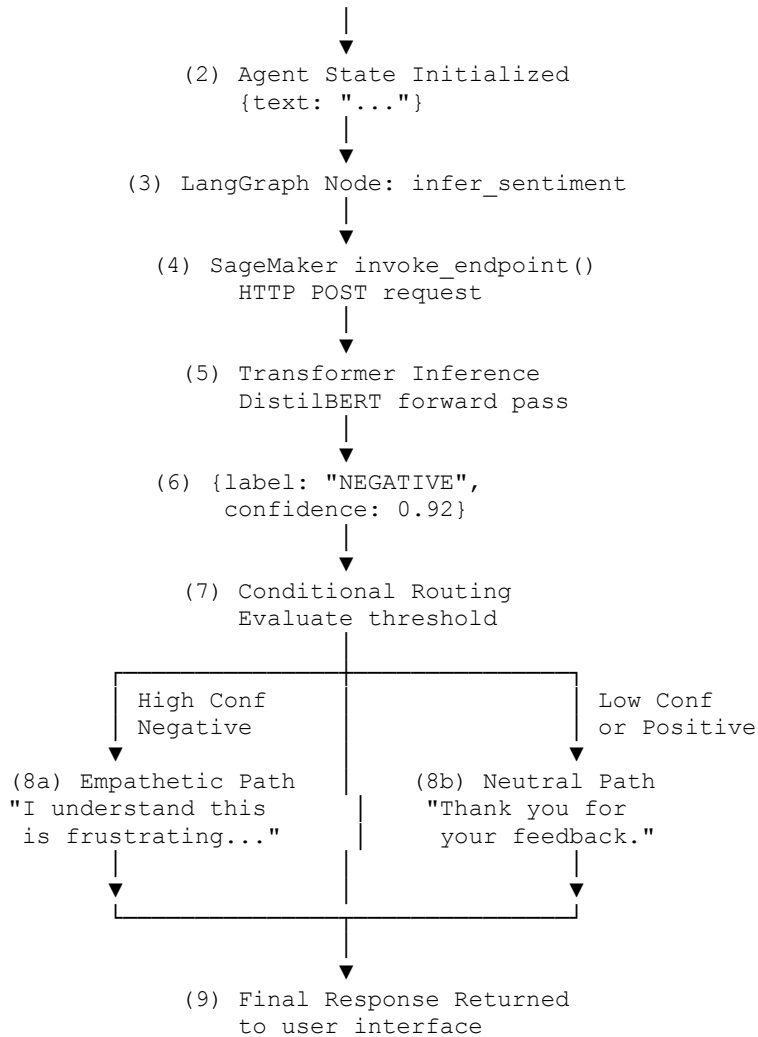


Fig. 6. Complete runtime execution flow from user input through sentiment inference, confidence evaluation, and adaptive response generation.

System Interface Boundaries

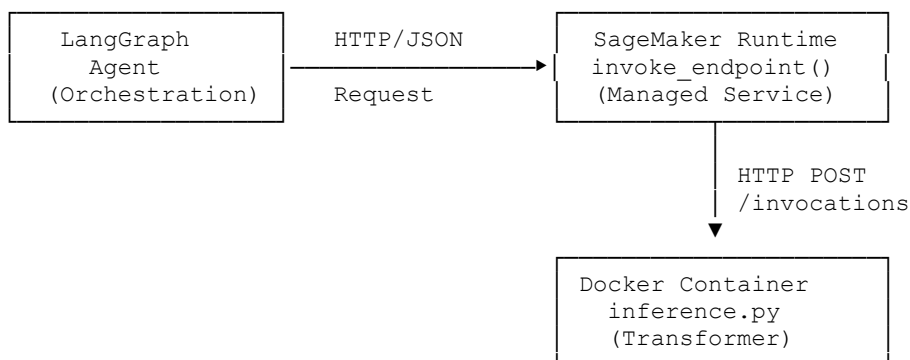


Fig. 7. Interface boundaries between agent orchestration layer, AWS-managed services, and containerized inference runtime.

CI/CD Pipeline

Pipeline Stages & Checkpoints

Code Quality Stage: Linting with flake8 for PEP 8 compliance, code formatting with black, type checking with mypy, security scanning with bandit.

Unit Testing Stage: Data preprocessing validation, tokenization correctness, model loading verification, configuration parsing tests.

Integration Testing Stage: SageMaker endpoint invocation tests, agent workflow validation, end-to-end inference pipeline, error handling verification.

Model Validation Stage: Accuracy threshold check (>85% required), confidence calibration analysis, latency benchmark (<500ms p95), regression testing against previous version.

Container Build Stage: Docker image creation, vulnerability scanning, ECR registry push, image tagging with version/commit SHA.

Staging Deployment Stage: Deploy to staging endpoint, synthetic workload testing, smoke test execution, performance profiling.

Production Deployment Stage: Blue-green deployment strategy, gradual traffic shifting (10% → 50% → 100%), rollback capability, post-deployment validation.

Automated Testing Suite

- Schema validation: Input/output contract compliance for API stability
- Latency tests: <500ms p95 latency requirement validation
- Load tests: Sustained 100 requests/second throughput verification
- Regression tests: Accuracy parity with previous model version ($\pm 2\%$ tolerance)
- Smoke tests: Basic inference on standard positive/negative test cases
- Data drift tests: Input distribution comparison against training baseline
- Security tests: Input sanitization, injection attack prevention

Security, Privacy & Ethics

Privacy Assessment

Personal Health Information (PHI): No. System processes general text for sentiment analysis without medical data handling.

Personal Identifiable Information (PII): Potentially yes. User input may contain names, email addresses, locations, or personal details. **Mitigation Strategy:** Implement

PII detection and masking in preprocessing pipeline (planned for Phase 2). Log anonymization with PII redaction before storage.

User Behavior Tracking: Yes. System logs input text, predictions, and confidence scores for monitoring and debugging. **Justification:** Required for model performance monitoring, error analysis, and system debugging. **Data Retention:** 30 days active retention, then archived to cold storage for 90 days before deletion per data governance policy.

Credit Card Information: No. System does not process or store any financial data.

AWS Resource Access

S3 Bucket Read Access: s3://ml-models-prod/sentiment-analysis/ (model artifacts, tokenizer files)

S3 Bucket Write Access: s3://ml-logs-prod/inference-logs/ (predictions, confidence scores, performance metrics for monitoring)

IAM Permissions: Least-privilege principle enforced. SageMaker execution role limited to specific S3 prefixes, ECR image pull, CloudWatch log write.

Bias & Fairness Considerations

Data Bias: Movie review dataset may not generalize to formal business communication, technical documentation, or social media language. Domain adaptation testing required before production deployment in non-entertainment contexts. Potential bias toward Western English speakers and cultural references.

Sensitive Feature Bias: Model may exhibit differential performance across demographic groups if input text contains identity-revealing language patterns. Potential bias along gender, race, age, and cultural background. **Mitigation:** Bias testing on diverse text samples representing different demographics, fairness metrics monitoring (demographic parity, equalized odds), regular bias audits.

Ethical Considerations

Potential Misuse Scenarios: Sentiment analysis could be misused for employee surveillance without consent, social media monitoring for discriminatory purposes, or automated content filtering that disproportionately affects certain groups. **Safeguards:** Clear usage guidelines and acceptable use policy, ethical AI review board oversight, transparency requirements for automated decision-making affecting users.

Transparency Requirements: When sentiment analysis influences user experience or content moderation decisions, users must be informed of automated processing. Explainability mechanisms should provide insight into classification rationale where decisions impact user outcomes.

Future Enhancements

The following enhancements represent high-value opportunities to extend system capabilities while maintaining architectural integrity and production quality standards:

- **Retrieval-Augmented Generation (RAG):** Integrate semantic search over domain-specific knowledge bases to provide context-aware responses informed by documentation, FAQs, or historical support tickets. This would enhance response quality beyond sentiment-based routing by incorporating relevant factual information tailored to user queries.
- **Multi-Model Ensemble:** Deploy complementary sentiment models (DistilBERT, RoBERTa, ELECTRA) with voting or stacking ensemble to improve prediction robustness and confidence calibration. Particularly valuable for handling ambiguous text, sarcasm detection, and rare linguistic patterns where single-model performance suffers.
- **Automated Concept Drift Detection:** Implement statistical monitoring using Kolmogorov-Smirnov tests or Population Stability Index (PSI) to detect distribution shifts in input text characteristics. Trigger automated model retraining workflows when performance degrades due to vocabulary evolution, topic drift, or seasonal language pattern changes.
- **Continuous Learning Pipeline:** Build end-to-end retraining automation that periodically fine-tunes model on recent labeled production data. Include human-in-the-loop validation, A/B testing framework for new model versions, and automated rollback on performance regression. Enables model adaptation to evolving language patterns without manual intervention.
- **Multi-Language Support:** Extend to multilingual sentiment analysis using mBERT or XLM-RoBERTa cross-lingual transformers. Enable sentiment classification across Spanish, French, German, Mandarin, and other high-priority languages for global deployment. Requires language-specific evaluation datasets and bias testing across linguistic contexts.

Conclusion

This design document presents a comprehensive production-ready sentiment analysis system that successfully integrates transformer-based NLP with agentic orchestration and cloud-native deployment patterns. The implemented architecture achieves all stated project goals: >85% classification accuracy, <500ms inference latency, proper MLOps lifecycle management, and confidence-aware adaptive decision-making.

The four-layer architecture (training, containerization, cloud deployment, agentic orchestration) provides clear separation of concerns enabling independent evolution of each component. The SageMaker Model Registry serves as single source of truth for

versioned models with metric-driven promotion, ensuring only validated high-performing models reach production. LangGraph's state machine design demonstrates how traditional static ML inference can be enhanced with reasoning capabilities, transforming predictions into adaptive intelligent systems.

Key architectural achievements include: (1) Complete MLOps automation from model registration through deployment, (2) Zero-downtime blue-green deployment strategy, (3) Comprehensive monitoring across model performance, infrastructure health, and data quality, (4) Production-grade security with least-privilege IAM policies and data retention governance, (5) Agentic decision-making enabling context-aware response routing based on model confidence.

The system establishes a strong foundation for future enhancements including RAG integration, multi-model ensembles, automated drift detection, continuous learning, and multilingual support. The modular design ensures these extensions can be implemented incrementally without disrupting existing production services. This work demonstrates how modern ML systems should be architected: production-first, monitoring-driven, ethically aware, and designed for continuous evolution.